Orchestrator Architecture Assessment

Multi-Agent Orchestration & Skill Router Scalability

Health Sciences Skills - Cortex Code

Pressure Test Report - March 26, 2026

Scope: health-sciences-coco-skills-incubator repository
19 top-level skills | ~40 sub-skills | Twin orchestrator profiles
+ ISF Blueprint Routing Enhancement (hcls-cross-isf-reader)

Executive Summary

This assessment pressure-tests two core dimensions of the Health Sciences orchestrator architecture: (1) the multi-agent orchestration model for context window management and sub-tasking, and (2) the skill router table's ability to scale from the current 18 skills to 200+. It also evaluates a proposed enhancement: ISF-driven blueprint routing via a new hcls-cross-isf-reader skill that adds a Step 0 to the orchestrator, enabling solution-level plan assembly from pre-mapped blueprints with automated skill gap analysis.

The architecture is well-designed for its current phase and represents thoughtful engineering: template-driven generation prevents orchestrator drift, QA validation (12 checks) catches structural regressions, preflight patterns enable graceful degradation, and the Plan-then-Execute protocol gives the LLM a structured decision framework.

However, the system was designed as a static prompt engineering system. The critical finding is that it must evolve into a dynamic retrieval and dispatch system as skill count grows past approximately 50. Key risks include context window saturation from accumulated skill prompts, keyword collision in the routing table, and O(n-squared) growth in cross-domain pattern maintenance.

Artifacts Reviewed

- agents/health-sciences-incubator.md (orchestrator, ~430 lines)
- agents/health-sciences-solutions.md (production orchestrator, empty scaffold)
- templates/orchestrator.md.j2 (Jinja2 template, ~320 lines)
- templates/skills_incubator.yaml (registry, 344 lines, 18 skills)
- scripts/generate_orchestrators.py (twin-profile generator)
- scripts/qa_validate_orchestrator.py (12-check validation suite)
- HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md (developer guide)
- Representative SKILL.md files: imaging (router + 7 sub-skills), clinical-nlp (router + 15 sub-skills)
- [NEW] sfc-gh-jurrutia/isf-skill-hcls-workflow repo: hcls-cross-isf-reader SKILL.md, hcls-solutions.yaml (9 blueprints)
- [NEW] Updated orchestrator.md.j2 template with Step 0 ISF routing and gap analysis

Part 1: Multi-Agent Orchestrator Model

This section evaluates the orchestrator's approach to context window management, skill chaining, inter-skill communication, and sub-tasking.

1.1 What Is Working Well

- **Twin-orchestrator design** generated from a single Jinja2 template prevents structural drift between incubator and production profiles. The drift-check in generate_orchestrators.py catches unexpected divergences automatically.
- **Plan-then-Execute protocol** with mandatory user gates prevents runaway execution and gives the LLM a structured thinking framework. The lightweight skip for single-skill queries avoids unnecessary friction.
- **Preflight checks with graceful fallback** (READY/MISSING pattern) mean skills degrade gracefully instead of failing. CKEs and Data Model Knowledge services probe before use.
- **Platform affinities in SKILL.md frontmatter** decouple domain skills from platform skills declaratively. Domain skills declare what they produce and benefit from without hardcoding platform skill names in their body.
- **QA validation suite** (12 checks) is comprehensive: bidirectional registry-directory matching, taxonomy tree validation, CKE used_by references, platform affinity frontmatter checks, and overlap verification.

1.2 Critical Weaknesses

A. Context Window Saturation

The orchestrator .md alone is ~430 lines (~3,000 tokens). When Cortex Code loads this as the system prompt, every skill invocation adds that skill's SKILL.md to the context. A 5-step Cross-Domain Pattern like 'Clinical Document Intelligence' chains: clinical-docs (router, ~200 lines) -> clinical-nlp (router, ~150+ lines) -> cke-pubmed -> cdata-fhir -> data-governance.

This burns 2,000-4,000 tokens on the orchestrator plus 1,000-3,000 tokens per skill before any user code, SQL, or tool output enters the context. With today's 18 skills this is survivable. At 50+ skills the orchestrator prompt alone risks exceeding useful working context.

B. No Context Eviction Strategy

Once a skill is loaded, it stays in the conversation context for the remainder of the session. There is no mechanism to 'unload' skill instructions after a step completes. A 5-step plan accumulates all 5 skill prompts even though only the current step is relevant. This is a fundamental LLM limitation, but the architecture designs no mitigation such as summarizing prior steps or truncating completed skill prompts.

C. Serial Skill Chaining, Not True Multi-Agent

The skill tool loads a prompt into the current conversation context. It does not spawn an isolated sub-agent with its own context window. A 5-skill pipeline runs in a single context with all 5 prompts stacked. A true multi-agent design would have an Orchestrator (owns the plan, manages state) dispatching to Skill Agent 1 (isolated context, returns structured output) then Skill Agent 2 (fresh context), and so on.

The current architecture is pseudo-multi-agent: it is really a single agent with dynamic prompt injection.

D. No Inter-Skill Contract or Interface

Skills produce Snowflake objects (tables, views), but there is no formal schema contract between them. When clinical-docs produces a table and clinical-nlp consumes it, the column names and types are implicitly assumed. At scale, this becomes fragile: a change to one skill's output can silently break downstream consumers.

E. Router-Within-Router Decision Depth

The pattern of router skills containing sub-skills (e.g., clinical-nlp has 17 intents, imaging has 7) creates a decision tree depth of 3: Orchestrator -> Router skill -> Sub-skill. The LLM must hold the orchestrator routing table, the router's intent table, and the sub-skill's instructions simultaneously. While this works at current scale, the compound routing error rate increases multiplicatively with depth.

1.3 Recommendations

Issue	Recommended Fix
Context bloat	Implement skill unloading: after a step completes, summarize its output into a structured JSON artifact and drop the skill prompt from active context.
Single-agent bottleneck	Evaluate CoCo's Task tool (subagents) for skill execution. Each skill runs in its own context window and returns a structured result to the orchestrator.
No inter-skill contracts	Define produces_schema and consumes_schema in SKILL.md frontmatter. Validate compatibility at generation time via the QA script.
Router depth	Cap routing at 2 levels (orchestrator -> skill). Flatten router sub-skills into top-level skills with the router becoming a disambiguation prompt, not a skill wrapper.

Part 2: Skill Router Table Scalability

This section evaluates whether the current keyword-trigger routing table, YAML registry, and Jinja2 template can scale from 18 to 200+ skills.

2.1 Current State

- 18 top-level skills, ~40 sub-skills across Provider, Pharma, and Cross-Industry
- All routing is a hardcoded Markdown table in the orchestrator (~120 lines)
- generate_orchestrators.py reads skills_incubator.yaml and renders via Jinja2
- QA validation covers 12 checks including bidirectional registry matching
- 9 cross-domain composition patterns, manually authored
- 4 overlap entries with manual disambiguation text

2.2 Why This Does Not Scale to 200 Skills

A. Prompt-Based Routing Hits Token Walls

At 200 skills x ~3 lines per skill, the routing table alone is ~600 lines (~4,000 tokens). Add sub-skills at a 2:1 ratio and the table reaches ~1,800 entries (~12,000 tokens). The LLM spends significant capacity just reading the routing table before doing any work. Worse, keyword-match accuracy degrades as trigger overlap increases.

B. Trigger Keyword Collision (Already Visible)

Even at 18 skills, collisions exist. The keyword 'extract' matches clinical-nlp (6 sub-skills), clinical-docs, FHIR, and pharmacovigilance. 'Adverse events' matches both pharmacovigilance and clinical-nlp/extraction-safety-care-planning. 'Clinical notes' matches clinical-nlp and clinical-docs.

At 200 skills, keyword disambiguation becomes combinatorially intractable for flat string matching. The current design has no scoring, weighting, or semantic similarity: it relies entirely on the LLM reading the table and picking the best match.

C. Monolithic YAML Registry

skills_incubator.yaml is a single 344-line file. At 200 skills it would reach approximately 4,000 lines. There is no sharding by domain, no per-team registries, and no auto-discovery from SKILL.md frontmatter.

D. Hardcoded Taxonomy in Jinja2 Template

The taxonomy structure in orchestrator.md.j2 directly references skill names using expressions like skills['hcls-provider-imaging'].name. Adding a new sub-industry (e.g., MedDevice with 4 skills) requires manual template editing. At 200 skills across 10+ sub-industries, this is unmaintainable.

E. Cross-Domain Patterns Are O(n-squared)

Each pattern is a manually authored step sequence referencing specific skills. At 200 skills, the number of possible composition patterns explodes. Manual authoring and maintenance of all valid patterns becomes infeasible.

F. Handcrafted Overlap Disambiguation

Today there are 4 overlaps with manual resolution text. At 200 skills, overlap detection and resolution must be automated rather than hand-maintained.

2.3 What a 200-Skill Architecture Needs

Approach	Description	Trade-offs
Semantic Router	Replace keyword triggers with embedding-based similarity. Each skill has an embedding; user query retrieves top-k skills.	Requires a Cortex Search Service over skill metadata. Adds ~200ms latency but eliminates keyword collision entirely.
Hierarchical Routing	Two-stage: (1) classify into sub-industry/domain (5-10 categories), (2) within that domain, retrieve matching skills (~20-30).	Mirrors existing taxonomy. Reduces prompt size by ~80%. First stage is a fast classifier.
Registry Sharding	Split skills_incubator.yaml into per-domain registries. Generator assembles from shards.	Better maintainability, enables per-domain ownership. Adds build complexity.
Auto-Discovery	Scan SKILL.md files at generation time; extract name, triggers, domain from frontmatter. Registry becomes derived.	Eliminates registry drift. Source of truth is the skill itself. But loses registry-only metadata.
Dynamic Loading	Orchestrator loads only a routing index (name + 1-line description). Full SKILL.md loaded only when selected.	Dramatically reduces base context. Requires lazy skill loading support in CoCo.
LLM-Generated Patterns	Instead of hardcoded patterns, the LLM composes skill chains dynamically from skill metadata (produces, consumes, affinities).	Eliminates O(n-squared) manual authoring. Risk of hallucinated compositions. Needs guardrails.

2.4 Recommended Phased Evolution

Phase 1: Now (18 skills)

Current design is adequate. Focus on adding inter-skill contracts (produces_schema / consumes_schema) and automated overlap detection to the QA validation script.

Phase 2: At 50 skills

Implement hierarchical routing. Stage 1 classifier routes to a domain (5-10 choices). Stage 2 loads only that domain's routing table. Split the registry YAML by domain (per-domain YAML shards).

Phase 3: At 100+ skills

Build a semantic router using Cortex Search over skill metadata. The orchestrator prompt shrinks to: Query the skill search service, retrieve top-3 matches, validate against the taxonomy, then invoke. Add auto-discovery from SKILL.md to eliminate registry drift.

Phase 4: At 200+ skills

Move to dynamic skill loading with a manifest index. The orchestrator becomes a lightweight dispatcher that never holds more than 1-2 skill prompts in context simultaneously. Patterns become LLM-generated from skill metadata graphs (produces/consumes/affinity edges).

Part 3: ISF Blueprint Routing Enhancement

A new skill, hcls-cross-isf-reader, has been proposed to bridge Snowflake's Industry Solutions Framework (ISF) with the HCLS skill orchestrator. This section assesses the enhancement's architecture, its impact on the routing model, and scalability implications.

3.1 What the Enhancement Adds

New Skill: hcls-cross-isf-reader

A Cross-Industry skill hosted at github.com/sfc-gh-jurrutia/isf-skill-hcls-workflow. It queries ISF via Cortex Search (solution discovery) and Cortex Analyst (solution detail) filtered to Healthcare & Life Sciences, then matches results against a local YAML blueprints file (references/hcls-solutions.yaml) that maps 9 known ISF HLS solutions to specific HCLS skill collections.

9 Pre-Mapped Solution Blueprints

Blueprint	Skill Collection	Pattern
Population Health Analytics	cdata-fhir, cdata-omop, claims, cke-pubmed	Real-World Evidence Study
Clinical Document Intelligence	clinical-docs, clinical-nlp, cke-pubmed, cdata-fhir	Clinical Document Intelligence
Drug Safety Signal Detection	pharmacovigilance, cke-pubmed, clinical-nlp, claims	Drug Safety Signal Detection
Medical Imaging Analytics	imaging, cdata-fhir, clinical-nlp, cke-pubmed	Imaging + Clinical Integration
Genomics + Clinical Outcomes	nextflow, variant-annotation, survival-analysis	Genomics + Clinical Outcomes
Single-Cell Analysis	single-cell-qc, scvi-tools	Single-Cell Analysis Pipeline
Clinical Trial Protocol	research-problem, cke-trials, cke-pubmed, protocol, survival	Clinical Trial Design
Lab Data Modernization	lab-allotrope	Lab Data Modernization
Real-World Evidence	claims, cke-trials, cdata-omop, survival, cke-pubmed	Real-World Evidence Study

Updated Orchestrator Routing: Step 0

The orchestrator template gains a new Step 0 before the existing Steps 1-4. When a user describes a business problem (not a specific technical task), Step 0 fires first, invokes the ISF reader, and presents three options:

- Plan A: Blueprint match found -- deploy the exact pre-mapped skill collection as-is
- Plan B: No match -- fall through to normal sub-industry/task/CKE routing (Steps 1-4)
- Plan C: Partial match -- start with the blueprint, customize from there

After plan assembly (whether from blueprint or routing), the ISF reader runs a skill gap analysis: it compares ISF pain points and use cases against the skill collection and flags any requirement that has no matching HCLS skill, suggesting $hcls-cross-skill-development for gaps.

3.2 Architectural Assessment

Strengths

- **Solution-level routing** bridges the gap between business problems and technical skills. Instead of the user needing to know 'I need FHIR + OMOP + claims,' they say 'I need population health analytics' and get a deterministic skill collection. This is the right abstraction layer.
- **Blueprint-as-code (YAML)** makes the mapping auditable, version-controlled, and diffable. New blueprints are a YAML append, not a template edit. This scales linearly with solutions.
- **Automated gap analysis** turns the orchestrator from a passive router into an active consultant. Surfacing ISF

pain points with no matching skill creates a clear backlog for skill development.

- **Graceful degradation** (Plan A/B/C pattern) means ISF availability issues never block the user. If ISF is down or no blueprint matches, the orchestrator falls through to existing routing transparently.
- **Cortex Search + Cortex Analyst dual-engine** approach for ISF querying is resilient. Fuzzy matching via Search for discovery, structured Analyst queries for detail, with automatic fallback between them.

Risks and Concerns

- **Context window cost of Step 0:** The ISF reader SKILL.md is ~290 lines. Loading it for Step 0 adds ~2,000 tokens before any domain skill is invoked. For simple technical tasks ('parse this DICOM file'), Step 0 is pure overhead. The 'when to run Step 0' heuristic relies on the LLM distinguishing business problems from technical tasks -- a fuzzy boundary.
- **ISF availability dependency:** Step 0 queries SUPPORT.ISF.ISF_SOLUTIONS_SEARCH and SUPPORT.ISF.ISF_ONTOLOGY_SEMANTIC_VIEW. If the ISF Cortex Search or Analyst services are unavailable (permissions, region, outage), the skill falls back to Plan B. But it burns context and latency attempting the queries before failing. A preflight probe (like CKEs have) would short-circuit faster.
- **Blueprint staleness:** hcls-solutions.yaml is a static file. If ISF solutions evolve (names change, new solutions added) and the YAML is not updated, the blueprint match rate degrades silently. There is no automated validation that blueprint names still match ISF solution names.
- **Gap analysis accuracy:** Matching ISF pain points/use cases to HCLS skills is done by the LLM at runtime, not deterministically. The gap analysis quality depends on how well ISF use case descriptions align with HCLS skill trigger keywords. At scale, false negatives (missed coverage) and false positives (spurious gaps) will increase.
- **Cross-repo coordination:** The ISF reader lives in a separate repo (sfc-gh-jurrutia/isf-skill-hcls-workflow) while the orchestrator template and registry live in the incubator repo. Blueprint YAML references skill names from the incubator. A skill rename in one repo silently breaks the other.

3.3 Recommendations

Issue	Recommended Fix
Context overhead	Add a preflight probe to Step 0 (like CKEs). If ISF services are unreachable, skip immediately. Also consider making Step 0 a subagent call (Task tool) so its context is isolated.
Blueprint staleness	Add a QA check that validates hcls-solutions.yaml blueprint names against ISF solution names (via Cortex Search). Run as CI or pre-commit hook.
Cross-repo drift	Add a blueprint validation check to qa_validate_orchestrator.py that reads hcls-solutions.yaml and verifies every skill name exists in skills_incubator.yaml.
Gap analysis accuracy	Pre-compute a mapping from ISF use case IDs to HCLS skill names in the blueprint YAML (deterministic). Use LLM only for unmapped use cases.
Scalability to 50+ ISF solutions	Embed blueprint metadata into a Cortex Search service (like the CKEs). Replace YAML file scan with semantic retrieval over blueprints.

3.4 Revised Routing Flow

The following shows the updated orchestrator routing sequence with Step 0 integrated. The existing Steps 1-4 are unchanged; Step 0 is additive and optional.

Step	Name	Description
Step 0 (NEW)	ISF Blueprint Check	If user describes a business problem, invoke hcls-cross-isf-reader. Query ISF, match blueprints, present Plan A/B/C. Run skill gap analysis.
Step 1	Route by Sub-Industry	Classify: Provider / Pharma / Payer. (Skipped if Step 0 returned Plan A or C.)
Step 2	Route by Task	If sub-industry ambiguous, route by task type. (Skipped if Step 0 returned Plan A or C.)
Step 3	Cross-Industry Skills	Augment plan with CKEs, research-problem-selection, ISF reader, skill-development.
Step 4	Accept Overlaps	Resolve skills that serve multiple sub-industries.
Plan Gate	Present to User	Show assembled plan. Wait for explicit approval before executing.
Gap Report	Skill Coverage	List ISF requirements vs matched skills. Flag gaps. Suggest skill-development.

Summary Scorecard

The following scorecard grades each architectural dimension at the current scale (18 skills) and projects the grade if the skill count reaches 200 without architectural changes.

Dimension	Current (18)	At 200 Skills
Routing accuracy	B+	D (keyword collision)
Context efficiency	B-	F (prompt overflow)
Maintainability	B	D (monolith registry)
Inter-skill contracts	D	F (implicit assumptions)
Skill chaining / isolation	C	D (single context window)
QA / validation	A-	B- (checks need scaling)
Governance guardrails	A	A (cross-cutting, survives)
Pattern composition	B+	D (O(n^2) manual authoring)
ISF blueprint routing (NEW)	A-	B (YAML staleness risk)
Skill gap analysis (NEW)	B+	C+ (LLM accuracy at scale)

> *Bottom line: The architecture is well-designed for its current phase. The ISF blueprint enhancement (Part 3) is the right abstraction -- it lifts routing from technical skill matching to business solution matching. The critical evolution path remains: static prompt engineering -> dynamic retrieval + dispatch as skills grow past ~50.*

Appendix: Current Architecture Snapshot

A. File Layout

File	Purpose
agents/health-sciences-incubator.md	Generated orchestrator (incubator profile)
agents/health-sciences-solutions.md	Generated orchestrator (production, empty scaffold)
templates/orchestrator.md.j2	Jinja2 template (~320 lines)
templates/skills_incubator.yaml	Incubator skill registry (18 skills, 344 lines)
templates/skills_production.yaml	Production registry (empty scaffold)
scripts/generate_orchestrators.py	Twin-profile generator with drift detection
scripts/qa_validate_orchestrator.py	12-check validation suite
skills/hcls-*/SKILL.md	18 top-level skill definitions

B. Skill Inventory

18 top-level skills organized across 3 sub-industries and cross-industry:

Skill	Type	Domain
hcls-provider-imaging	Router (7)	Provider > Clinical Research
hcls-provider-imaging-dicom-parser	Standalone	Provider > Clinical Research
hcls-provider-cdata-fhir	Standalone	Provider > Clinical Data Mgmt
hcls-provider-cdata-clinical-nlp	Router (15)	Provider > Clinical Data Mgmt
hcls-provider-cdata-clinical-docs	Router (5)	Provider > Clinical Data Mgmt
hcls-provider-cdata-omop	Standalone	Provider > Clinical Data Mgmt
hcls-provider-claims-data-analysis	Standalone	Provider > Revenue Cycle
hcls-pharma-dsafety-pharmacovigilance	Standalone	Pharma > Drug Safety
hcls-pharma-dsafety-clinical-trial-protocol	Standalone	Pharma > Drug Safety
hcls-pharma-genomics-nextflow	Standalone	Pharma > Genomics
hcls-pharma-genomics-variant-annotation	Standalone	Pharma > Genomics
hcls-pharma-genomics-single-cell-qc	Standalone	Pharma > Genomics
hcls-pharma-genomics-scvi-tools	Standalone	Pharma > Genomics
hcls-pharma-genomics-survival-analysis	Standalone	Pharma > Genomics
hcls-pharma-lab-allotrope	Standalone	Pharma > Lab Operations
hcls-cross-research-problem-selection	Standalone	Cross-Industry
hcls-cross-cke-pubmed	CKE	Cross-Industry
hcls-cross-cke-clinical-trials	CKE	Cross-Industry
hcls-cross-isf-reader (NEW)	ISF Bridge	Cross-Industry

C. QA Validation Checks

Check	Description
1	Every $ref in orchestrator has a matching SKILL.md
2	Every top-level SKILL.md is referenced in orchestrator
3	Folder name matches SKILL.md name (top-level)
4	Imaging sub-skills exist in filesystem
5	Taxonomy tree entries match filesystem directories
6	Reference consistency (count per skill across orchestrator)
7	Standalone skills verification
8	Twin orchestrator drift detection

9	Registry vs directories bidirectional match
10	Platform affinities frontmatter validation
11	CKE used_by reference validation
12	Overlap entries reference validation